بسم الله الرحمن الرحيم

King Abdulaziz University
Faculty of Computing and information Technology
Computer Science Department

CPCS-214 Computer Architecture and Organization

23 - Performance Factors

CPU Performance Factors

- A simple formula relates the most basic metrics (clock cycles and clock cycle time) to CPU time:

$$\text{CPU execution time for a program} = \text{CPU clock cycles for a program} \times \text{Clock cycle time}$$

CPU Performance Factors

- A simple formula relates the most basic metrics (clock cycles and clock cycle time) to CPU time:

$$\text{CPU execution time for a program} = \text{CPU clock cycles for a program} \times \text{Clock cycle time}$$

- Alternatively, because *CR* and *CCT* are inverses

$$\text{CPU execution time for a program} = \frac{\text{CPU clock cycles for a program}}{\text{Clock rate}}$$

CPU Performance Factors

- HW designer can improve performance by reducing
  - Number of clock cycles required for a program or
  - Length of clock cycle
- Designer often faces a trade-off between number of clock cycles needed for a program and the length of each cycle
  - Many techniques that decrease *CC* may also increase *CCT*

Example

- A program runs in 10 seconds on computer A, which has a 2 GHz clock. A designer is required to build computer, B, which will run this program in 6 seconds. The designer has determined that an increase in *CR* is possible, but this increase will affect rest of CPU design, causing computer B to require 1.2 times as many *CCs* as computer A for this program. What *CR* should the designer target?

Example (cont.)

- First find number of clock cycles required for the program on A:

$$\text{CPU time}_A = \frac{\text{CPU clock cycles}_A}{\text{Clock rate}_A}$$

Example (cont.)

- First find number of clock cycles required for the program on A:

$$\text{CPU time}_A = \frac{\text{CPU clock cycles}_A}{\text{Clock rate}_A}$$

$$10 \text{ seconds} = \frac{\text{CPU clock cycles}_A}{2 \times 10^9 \frac{\text{cycles}}{\text{second}}}$$

Example (cont.)

- First find number of clock cycles required for the program on A:

$$\text{CPU time}_A = \frac{\text{CPU clock cycles}_A}{\text{Clock rate}_A}$$

$$10 \text{ seconds} = \frac{\text{CPU clock cycles}_A}{2 \times 10^9 \frac{\text{cycles}}{\text{second}}}$$

$$\text{CPU clock cycles}_A = 10 \text{ seconds} \times 2 \times 10^9 \frac{\text{cycles}}{\text{second}} = 20 \times 10^9 \text{ cycles}$$

Example (cont.)

- CPU time for B can be found using this equation:

$$\text{CPU time}_B = \frac{1.2 \times \text{CPU clock cycles}_A}{\text{Clock rate}_B}$$

Example (cont.)

- CPU time for B can be found using this equation:

$$\text{CPU time}_B = \frac{1.2 \times \text{CPU clock cycles}_A}{\text{Clock rate}_B}$$

$$6 \text{ seconds} = \frac{1.2 \times 20 \times 10^9 \text{ cycles}}{\text{Clock rate}_B}$$

Example (cont.)

- CPU time for B can be found using this equation:

$$\text{CPU time}_B = \frac{1.2 \times \text{CPU clock cycles}_A}{\text{Clock rate}_B}$$

$$6 \text{ seconds} = \frac{1.2 \times 20 \times 10^9 \text{ cycles}}{\text{Clock rate}_B}$$

$$\text{Clock rate}_B = \frac{1.2 \times 20 \times 10^9 \text{ cycles}}{6 \text{ seconds}} = \frac{0.2 \times 20 \times 10^9 \text{ cycles}}{\text{second}} = \frac{4 \times 10^9 \text{ cycles}}{\text{second}} = 4 \text{ GHz}$$

Example (cont.)

- CPU time for B can be found using this equation:

$$\text{CPU time}_B = \frac{1.2 \times \text{CPU clock cycles}_A}{\text{Clock rate}_B}$$

$$6 \text{ seconds} = \frac{1.2 \times 20 \times 10^9 \text{ cycles}}{\text{Clock rate}_B}$$

$$\text{Clock rate}_B = \frac{1.2 \times 20 \times 10^9 \text{ cycles}}{6 \text{ seconds}} = \frac{0.2 \times 20 \times 10^9 \text{ cycles}}{\text{second}} = \frac{4 \times 10^9 \text{ cycles}}{\text{second}} = 4 \text{ GHz}$$

- To run program in 6 seconds, B must have twice clock rate of A.

Instruction Performance

- Previous performance equations did not include any reference to the number of instructions needed for the program.
- Compiler generates instructions to execute, and computer had to execute instructions to run program
  - Execution time must depend on number of instructions in a program

Instruction Performance

- One way to think about execution time is that it equals the number of instructions executed multiplied by the average time per instruction
- So number of *CCs* required for a program can be written as

$$\text{CPU clock cycles} = \text{Instructions for a program} \times \begin{matrix}\text{Average clock cycles} \\ \text{per instruction}\end{matrix}$$

Cycles Per Instruction

- Average number of clock cycles each instruction takes to execute, abbreviated as *CPI*
- Since different instructions may take different amounts of time depending on what they do
  - *CPI* is an average of all instructions executed in program
- *CPI* provides way of comparing two different implementations of same ISA
  - Since number of instructions executed for a program will be the same

Example

- We have two implementations of same ISA. Computer A has a *CCT* of 250 ps and a *CPI* of 2.0 for a program, and computer B has a *CCT* of 500 ps and a *CPI* of 1.2 for the same program. Which computer is faster and by how much?

Example (cont.)

- Each computer executes the same number of instructions for the program; call this number *I*
- Find number of processor clock cycles for each computer:

$$\text{CPU clock cycles}_A = I \times 2.0$$
$$\text{CPU clock cycles}_B = I \times 1.2$$

Example (cont.)

- Now we can compute the CPU time for each computer:

$$\begin{aligned}\text{CPU time}_\text{A} &= \text{CPU clock cycles}_\text{A} \times \text{Clock cycle time} \\ &= I \times 2.0 \times 250 \text{ ps} = 500 \times I \text{ ps}\end{aligned}$$

- Likewise, for B:

$$\text{CPU time}_\text{B} = I \times 1.2 \times 500 \text{ ps} = 600 \times I \text{ ps}$$

- Clearly, computer A is faster.

Example (cont.)

- Amount faster is given by the ratio of the execution times:

$$\frac{\text{CPU performance}_A}{\text{CPU performance}_B} = \frac{\text{Execution time}_B}{\text{Execution time}_A} = \frac{600 \times I\,\text{ps}}{500 \times I\,\text{ps}} = 1.2$$

- Computer A is 1.2 times as fast as computer B for this program.

Classic CPU Performance Equation

- Performance equation in terms of instruction count (the number of instructions executed by the program), CPI, and clock cycle time:

$$\text{CPU time} = \text{Instruction count} \times \text{CPI} \times \text{Clock cycle time}$$

Classic CPU Performance Equation

- Performance equation in terms of instruction count (number of instructions executed by program), CPI, and clock cycle time:

$$\text{CPU time} = \text{Instruction count} \times \text{CPI} \times \text{Clock cycle time}$$

- Since clock rate is the inverse of clock cycle time:

$$\text{CPU time} = \frac{\text{Instruction count} \times \text{CPI}}{\text{Clock rate}}$$

Example

- A compiler designer is to decide between two code sequences for a particular computer. HW designers have supplied the following facts:

	CPI for each instruction class		
	A	B	C
CPI	1	2	3

- For a particular high-level language statement, the compiler writer is considering two code sequences that require the following instruction counts:

Code sequence	Instruction counts for each instruction class		
	A	B	C
1	2	1	2
2	4	1	1

- Which code sequence executes the most instructions? Which will be faster? What is the CPI for each sequence?

Example (cont.)

- Sequence 1 executes 2 + 1 + 2 = 5 instructions
- Sequence 2 executes 4 + 1 + 1 = 6 instructions
- Therefore, sequence 1 executes fewer instructions.
- Use equation for CPU clock cycles based on *IC* and *CPI* to find the total number of clock cycles for each sequence:

$$\text{CPU clock cycles} = \sum_{i=1}^{n} (\text{CPI}_i \times \text{C}_i)$$

- This yields

$$\text{CPU clock cycles}_1 = (2 \times 1) + (1 \times 2) + (2 \times 3) = 2 + 2 + 6 = 10 \text{ cycles}$$

$$\text{CPU clock cycles}_2 = (4 \times 1) + (1 \times 2) + (1 \times 3) = 4 + 2 + 3 = 9 \text{ cycles}$$

Example (cont.)

- So code sequence 2 is faster, even though it executes one extra instruction. Since code sequence 2 takes fewer overall clock cycles but has more instructions, it must have a lower CPI
- CPI values can be computed by

$$\text{CPI} = \frac{\text{CPU clock cycles}}{\text{Instruction count}}$$

$$\text{CPI}_1 = \frac{\text{CPU clock cycles}_1}{\text{Instruction count}_1} = \frac{10}{5} = 2.0$$

$$\text{CPI}_2 = \frac{\text{CPU clock cycles}_2}{\text{Instruction count}_2} = \frac{9}{6} = 1.5$$

Execution Time

- We can see how these factors are combined to yield execution time measured in seconds per program:

$$\text{Time} = \text{Seconds/Program} = \frac{\text{Instructions}}{\text{Program}} \times \frac{\text{Clock cycles}}{\text{Instruction}} \times \frac{\text{Seconds}}{\text{Clock cycle}}$$

Units of Performance Components

Components of performance	Units of measure
CPU execution time for a program	Seconds for the program
Instruction count	Instructions executed for the program
Clock cycles per instruction (CPI)	Average number of clock cycles per instruction
Clock cycle time	Seconds per clock cycle

Instruction Mix

- When comparing two computers, you must look at all three components, which combine to form execution time
- If some of the factors are identical, like *CR* in the previous example, performance can be determined by comparing all the nonidentical factors
- Since CPI varies by instruction mix, both *IC* and *CPI* must be compared, even if clock rates are identical

Program Performance

- Performance of a program depends on
  - Algorithm
  - Language
  - Compiler
  - Architecture
- Table summarizes how these components affect factors in CPU performance equation

Program Performance

Hardware or software component	Affects what?	How?
Algorithm	Instruction count, possibly CPI	The algorithm determines the number of source program instructions executed and hence the number of processor instructions executed. The algorithm may also affect the CPI, by favoring slower or faster instructions. For example, if the algorithm uses more divides, it will tend to have a higher CPI.
Programming language	Instruction count, CPI	The programming language certainly affects the instruction count, since statements in the language are translated to processor instructions, which determine instruction count. The language may also affect the CPI because of its features; for example, a language with heavy support for data abstraction (e.g., Java) will require indirect calls, which will use higher CPI instructions.
Compiler	Instruction count, CPI	The efficiency of the compiler affects both the instruction count and average cycles per instruction, since the compiler determines the translation of the source language instructions into computer instructions. The compiler's role can be very complex and affect the CPI in complex ways.
Instruction set architecture	Instruction count, clock rate, CPI	The instruction set architecture affects all three aspects of CPU performance, since it affects the instructions needed for a function, the cost in cycles of each instruction, and the overall clock rate of the processor.

Instructions Per Cycle

- Although you might expect that the minimum *CPI* is 1.0, some processors fetch and execute multiple instructions per *CC*
- To reflect that approach, some designers invert *CPI* to talk about *IPC*, or *instructions per clock cycle*
- If a processor executes on average 2 instructions per clock cycle, then it has an *IPC* of 2 and hence a *CPI* of 0.5

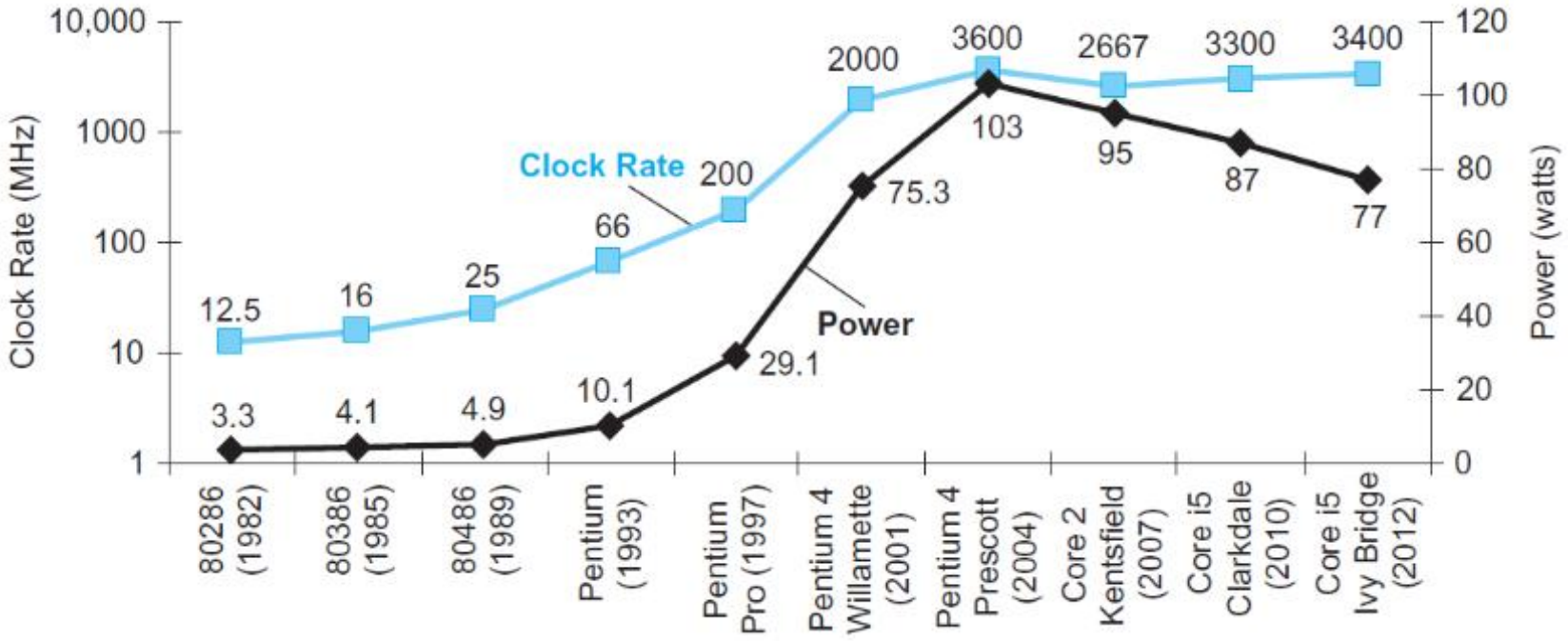
Turbo Mode

- Although *CCT* has traditionally been fixed, to save energy or temporarily boost performance, today's processors can vary their clock rates, so we would need to use the *average* clock rate for a program
- For example, the Intel Core i7 will temporarily increase clock rate by about 10% until the chip gets too warm
- Intel calls this *Turbo mode*

Power Wall

- Figure shows the increase in clock rate and power of eight generations of Intel microprocessors over 30 years
- Both clock rate and power increased rapidly for decades, and then fl attened off recently
- Reason they grew together is that they are correlated, and the reason for their recent slowing is that we have run into the
- practical power limit for cooling commodity microprocessors.

Power Wall


Clock Rate (MHz)
10,000
1000
100
10
1
Power (watts)
120
100
80
60
40
20
0
Clock Rate
Power
12.5
16
25
66
200
2000
3600
2667
3300
3400
3.3
4.1
4.9
10.1
29.1
75.3
103
95
87
77
80286 (1982)
80386 (1985)
80486 (1989)
Pentium (1993)
Pentium Pro (1997)
Pentium 4 Willamette (2001)
Pentium 4 Prescott (2004)
Core 2 Kentsfield (2007)
Core i5 Clarkdale (2010)
Core i5 Ivy Bridge (2012)

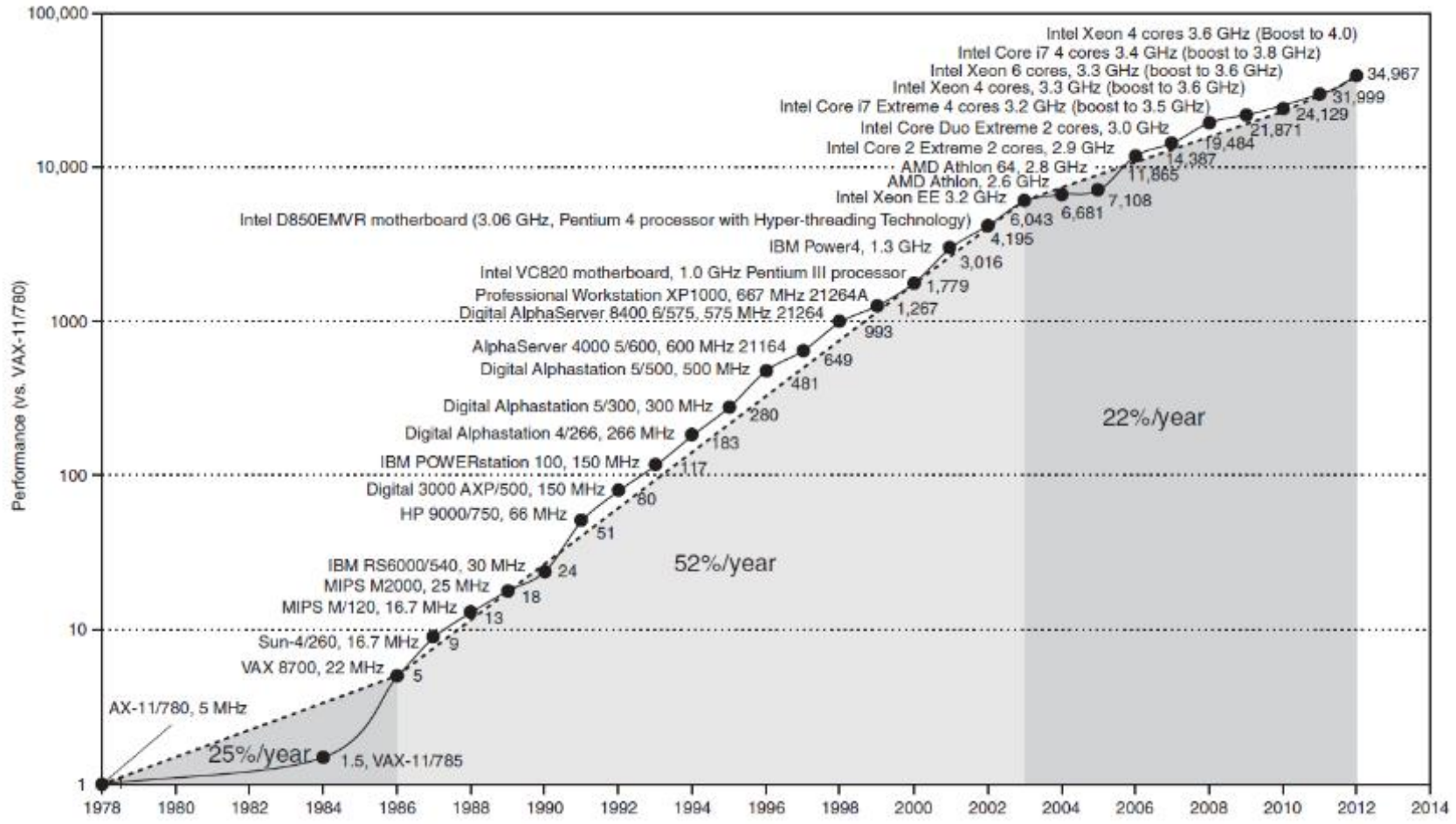
Energy

- Although power provides a limit to what we can cool, in PostPC Era critical resource is energy
- Battery life can trump performance in PMD,
- Architects of warehouse scale computers try to reduce costs of powering and cooling 100,000 servers as costs are high at this scale
- As measuring time in seconds is a safer measure of program performance than rate, energy metric joules is better measure than power rate like watts, which is just joules/second

Switch from Uniprocessors to Multiprocessors

- Power limit has forced a dramatic change in the design of microprocessors
- Figure shows the improvement in response time of programs for desktop microprocessors over time
- Since 2002, rate has slowed from factor of 1.5/year to 1.2/year

Switch from Uniprocessors to Multiprocessors



Multicores

- Rather than continuing to decrease response time of a single program running on single processor, companies are shipping microprocessors with multiple processors per chip, where benefit is often more on throughput than on response time
- To reduce confusion between processor and microprocessor, companies refer to processors as cores
  - microprocessors are generically called multicore microprocessors
- Quadcore microprocessor is a chip that contains four processors or four cores